LAB MID REPORT

SSDD

Submitted By:

Abdul Wadood [SP23-BCT-001]

Abdul Rehman [SP23-BCT-003]

Fiza Maheen [SP23-BCT-016]

Raveeha Naseem [FA22-BCT-045]

Date: 9 April 2026
Submitted To: Sir Ahmed Nawaz

DEPARTMENT OF COMPUTER SCIENCE [CYBER SECURITY]
COMSATS UNIVERSITY ISLAMABAD

1. Project Overview

The **Secure Online Voting System (CR Election 2026)** is a comprehensive DevSecOps project designed to demonstrate the integration of security at every stage of the Software Development Life Cycle (SDLC). In the modern digital era, the integrity of an election depends entirely on the robustness of the underlying software. This project serves as a prototype for a secure, transparent, and privacy-preserving voting platform.

2. Objective

The primary goal of this project is to move beyond functional development and implement **Security-by-Design**. Instead of treating security as a final step, we have embedded defensive mechanisms such as session protection, resource authorization, and hashed auditing directly into the application architecture.

3. The DevSecOps Approach

This project is structured into five distinct phases, moving from theoretical planning to automated security verification:

- **Planning & Requirements:** Identifying what needs to be protected (PNE).
- **Threat Modeling:** Predicting how an attacker might strike (STRIDE).
- **Secure Development:** Writing hardened code with active defenses.
- **Security Testing:** Verifying defenses through automated (SAST) and manual (DAST) testing.
- **Continuous Compliance:** Generating an audit trail for election transparency.

4. Problem Statement

Electronic voting systems face constant threats from malicious actors aiming to manipulate results, violate voter privacy, or disrupt the service. Common vulnerabilities like **IDOR**, **Cross-Site Request Forgery (CSRF)**, and **Clickjacking** can lead to illegitimate election outcomes. This project implements specific technical mitigations to neutralize these risks, ensuring a fair and verifiable election process.

Phase 1: Protection Need Elicitation

Name: Abdul-Rahman

Objective: The objective of this Protection Needs Elicitation (PNE) is to systematically identify and safeguard the critical assets of the Secure Voting System against potential threats. By analyzing risks such as spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege, the PNE ensures that appropriate protection measures are defined and implemented. The goal is to guarantee confidentiality of voter data, integrity of ballots and results, availability of the system during elections, strong authentication and authorization controls, and comprehensive auditability for transparency and accountability. This establishes a secure, trustworthy, and resilient voting environment that upholds democratic principles.

1. Assets

- Voter registration data
- Authentication credentials (MFA tokens)
- Ballots (encrypted votes)
- Vote counting results
- Audit logs

2. Threats

- **Spoofing:** fake user registration
- **Tampering:** vote manipulation
- **Repudiation:** denying vote cast
- **Information Disclosure**: vote leakage
- **Denial of Service:** voting system unavailability
- **Elevation of Privilege:** unauthorized admin access

3. Protection Needs

Confidentiality: encrypt all votes and user data

- **Integrity:** prevent vote tampering, ensure accurate counting
- **Availability:** system must remain online during voting
- **Authentication & Authorization:** enforce MFA and role-based access
- **Auditability:** maintain logs for transparency and accountability

Phase 2: Threat Modeling

Name: Fiza Maheen

Objective: To implement a comprehensive threat model for the **Secure Online Voting System** developed using Threat Modeling tool.

1. Introduction

This report presents a comprehensive threat model for the **Secure Online Voting System** developed using Threat Modeling tool. The model identifies potential security risks in the system architecture, data flows, and processes using industry-standard methodologies: **Data Flow Diagram (DFD)**, **STRIDE** threat categorization, **CVSS v3.1** risk scoring, and **Attack Tree** analysis.

The primary goal is to ensure the integrity, confidentiality, and availability of the voting process while protecting voter privacy and preventing election manipulation.

2. Data Flow Diagram (DFD)

The Data Flow Diagram illustrates how data moves between key entities in the Secure Voting System. It highlights trust boundaries, data stores, processes, and external entities.

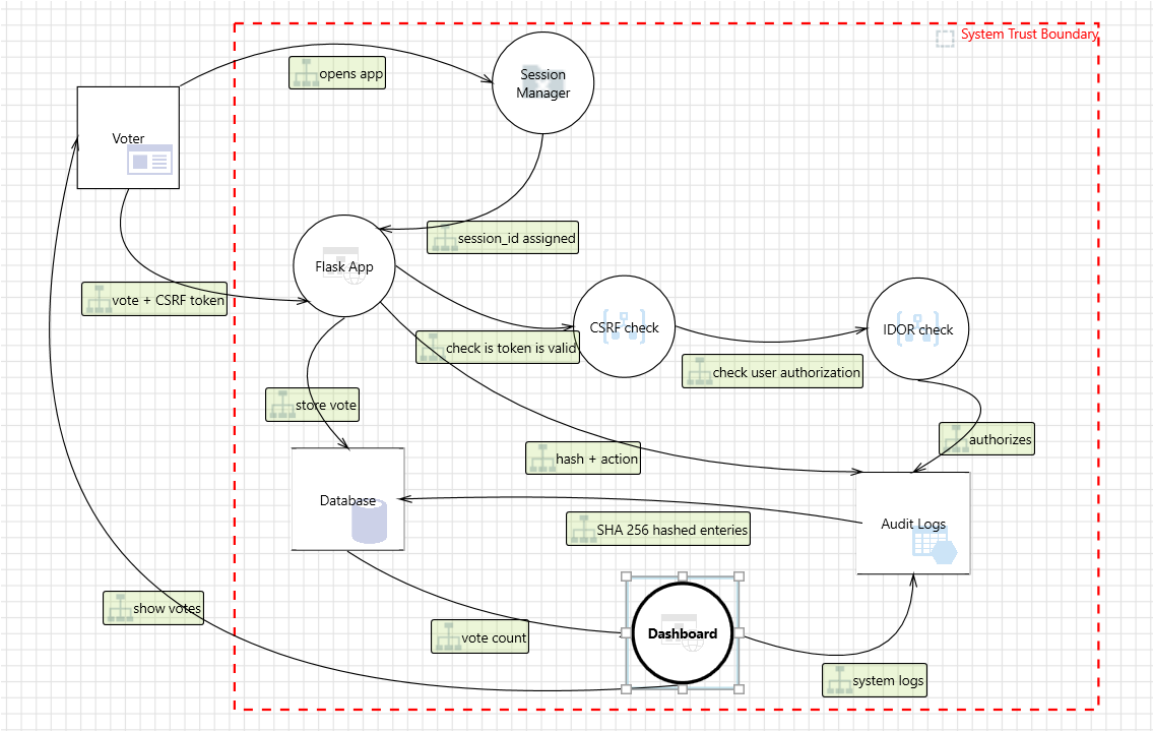
2.1 System Entities

- **Voter (External Entity)**
- **Flask App (Voting Application)**
- **Session Manager**
- **CSRF Check**
- **IDOR Check**
- **Database** (stores votes and voter data)
- **Audit Logs**
- **Dashboard** (Results / Vote Count)

2.2 Data Flows

- Voter opens the application → Session Manager assigns a session_id.
- Voter submits vote + CSRF token → Flask App.
- Flask App performs **CSRF check** and **IDOR check**.
- Valid vote is stored in the **Database**.
- Flask App sends (action + hash) → **Audit Logs** (using SHA-256 hashing).
- Dashboard fetches vote count and displays results from the **Database**.
- All actions are logged as system logs.

2.3 DFD Diagram



The DFD reveals critical trust boundaries between the Voter, Flask App, Database, Audit Logs, and Dashboard.

3. STRIDE Analysis

STRIDE is a threat modeling framework developed by Microsoft that categorizes threats into six types. The table below maps each category to relevant scenarios in the voting system along with mitigation strategies and severity.

STRIDE Category	Example in Voting System	Mitigation Strategy	Severity
Spoofing	Session hijacking or fake voter registration	Strong session management + MFA + proper IDOR/CSRF checks	High
Tampering	Altered votes in Database or forged CSRF token	Parameterized queries + SHA-256 hashing + integrity checks	Critical

STRIDE Category	Example in Voting System	Mitigation Strategy	Severity
Repudiation	User denies casting a vote	Tamper-evident Audit Logs with SHA-256 hashed entries	Medium
Information Disclosure	Leaked votes, session data, or hashed entries from logs	Encrypt sensitive data + secure error handling + restricted logs	High
Denial of Service	Overloading Flask App or Dashboard during election	Rate limiting + input validation	High
Elevation of Privilege	Bypassing IDOR check to access other voters' data	Strict RBAC + least privilege principle + proper authorization	Critical

4. CVSS v3.1 Risk Assessment

Each identified threat was scored using the **CVSS v3.1** base scoring methodology via the NVD CVSS Calculator.

Threat	CVSS v3.1 Score	Severity	Notes
Spoofing – Session Hijacking / Fake Voter	9.8	Critical	Allows fraudulent votes
Tampering – Vote Manipulation in Database	9.8	Critical	Direct impact on election integrity
Tampering – CSRF / IDOR Bypass	9.5	Critical	Unauthorized vote submission
Information Disclosure – Audit Logs / Errors	8.2	High	Violates voter confidentiality
Repudiation – Weak Audit Logs	6.5	Medium	Voter can deny actions
Denial of Service – App Overload	7.5	High	Disrupts voting process

Interpretation:

- Scores **9.0+ (Critical)** require immediate remediation.
- Scores **7.0–8.9 (High)** should be addressed before production deployment.

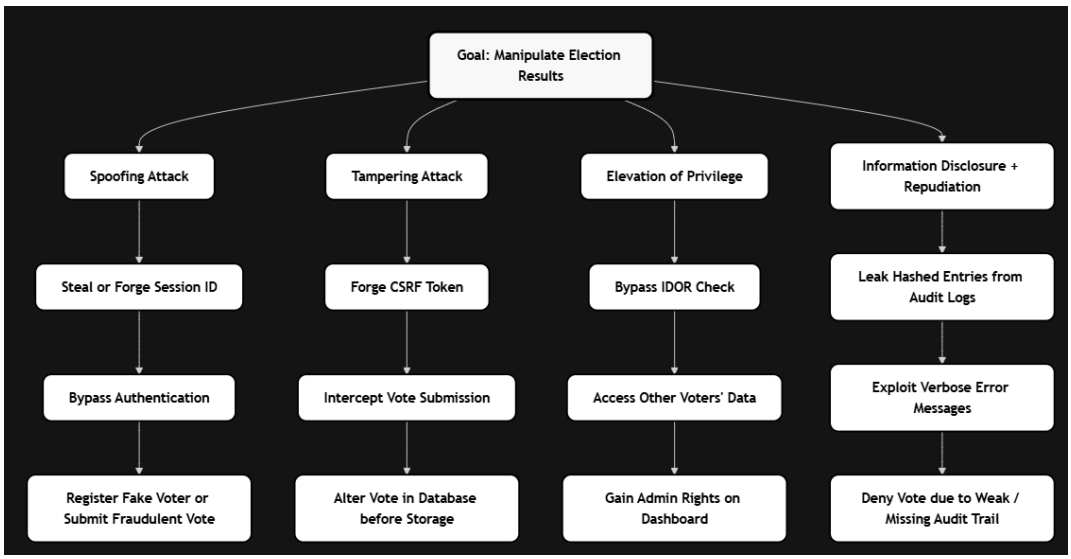
5. Attack Tree Analysis

The Attack Tree models the adversary's primary goal of **manipulating election results**, breaking it down into possible attack paths.

Root Goal: Manipulate Election Results

- **Path 1 – Spoofing** Attacker steals session_id or bypasses authentication → submits fake vote.
- **Path 2 – Tampering** Attacker forges CSRF token or tampers with vote data before storage in the Database.
- **Path 3 – Elevation of Privilege** Attacker bypasses IDOR check → accesses unauthorized voter data or the Dashboard.
- **Path 4 – Information Disclosure + Repudiation** Attacker leaks hashed entries from Audit Logs → denies actions due to weak logging.

5.1 Attack Tree Diagram



6. Summary and Recommendations

The DFD and Microsoft Threat Modeling Tool (TMT) analysis identified **103 threats**, confirming elevated risks particularly in authentication, vote submission, and logging components.

Most Critical Risks

- Session hijacking and vote tampering
- Bypassing CSRF/IDOR protection mechanisms
- Leakage of sensitive data through logs or error messages

Priority Mitigation Recommendations

1. Enforce **CSRF protection** and **IDOR checks** on all vote submission endpoints.
2. Use **parameterized queries** and **SHA-256 hashing** for all database operations and audit entries.
3. Implement robust **session management** with short expiry times, secure/HTTP-only cookies, and Multi-Factor Authentication (MFA) where feasible.
4. Make **Audit Logs tamper-evident** using cryptographic signing or hashing.
5. Encrypt all sensitive data **at rest** and **in transit** (enforce TLS 1.3).
6. Apply **rate limiting**, strict input validation, and output encoding.
7. Adopt **Role-Based Access Control (RBAC)** and the principle of least privilege.
8. Conduct regular **penetration testing** and security audits before any production deployment.

Implementing these controls will significantly strengthen the security posture of the Secure Voting System and help maintain trust in the electoral process.

Phase 3: Secure Implementation

Name: Raveeha Naseem

Objective: To implement a functional voting application with integrated security controls to mitigate IDOR, CSRF, and Clickjacking vulnerabilities.

1. Technology Stack

- **Backend:** Python 3.x with Flask Framework.
- **Security:** Flask-WTF for CSRF protection.
- **Logging:** Python logging module for secure audit trails.

2. Implementation Details :

A. CSRF Mitigation (Cross-Site Request Forgery)

Approach: We integrated the CSRFProtect middleware.

- **Implementation:** In app/__init__.py, we initialized the CSRF protection. In the frontend (vote.html), every form includes a hidden {{ csrf_token() }}.
- **Security Value:** This ensures that only requests originating from our actual website are processed, preventing attackers from tricking users into casting unauthorized votes.

app/__init__.py:

app > __init__.py > create_app

```python
from flask import Flask
from flask_wtf.csrf import CSRFProtect
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()
csrf = CSRFProtect()

def create_app():
    app = Flask(__name__)
    app.secret_key = "dev-secret-key-123" # Replace for production
    app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///voting.db'

    db.init_app(app)
    csrf.init_app(app)

    with app.app_context():
        from . import routes, security
        db.create_all()
        return app
```

app/templates/vote.html:

```
app > templates > vote.html > html > body > div.container.mt-5 > form > div.row.g-4.mb-4.text-center > div.col-md-4
 2  <html lang="en">
51  <body>
52      <div class="nav-header text-center">
53          <h1 class="fw-bold mb-1">SECURE ONLINE VOTING APP</h1>
54          <p class="slogan mb-0">Select your Representative: Shaping the Future of Our Class Together!</p>
55      </div>
56
57      <div class="container mt-5">
58          <form method="POST" action="/cast_vote">
59              <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
60
61              <h4 class="text-center text-white-50 mb-5">VOTING PORTAL: CR ELECTION 2026</h4>
62
63              <div class="row g-4 mb-4 text-center">
64                  <div class="col-md-4">
65                      <input type="radio" class="btn-check" name="candidate" id="c1" value="Ali" required>
66                      <label class="card candidate-card" for="c1">
67                          <div class="symbol-circle">A</div>
68                          <div class="candidate-name">Ali</div>
69                          <p class="small opacity-75">Visionary Leadership</p>
70                      </label>
```

B. IDOR Mitigation (Insecure Direct Object Reference)

Approach: We moved away from generic queries to **Scoped Queries**.

- **Implementation:** In routes.py, when a user requests a ballot receipt via /ballot/<id>, the system does not just search by ID. It searches using Ballot.query.filter_by(id=ballot_id, user_id=current_user).
- **Security Value:** Even if a user manually changes the ID in the URL, the system will return a 403 Unauthorized or 404 because the user_id won't match the active session.

app/routes.py

```
app > routes.py > get_ballot
37  def cast_vote():
49      db.session.commit()
50
51
52      logger.info(f"AUDIT: User [Hash: {user_hash}] has cast a vote for CR Election. Ballot ID: {new_vote.id}
53
54      return redirect(url_for('get_ballot', ballot_id=new_vote.id))
55
56  @app.route("/ballot/<int:ballot_id>")
57  def get_ballot(ballot_id):
58      user_id = session.get("user_id")
59      ballot = Ballot.query.filter_by(id=ballot_id, user_id=user_id).first()
60
61      if not ballot:
62          return "Unauthorized", 403
63
64      return render_template("thank_you.html", ballot=ballot)
```

C. Clickjacking Defense

Approach: Implementation of Secure HTTP Response Headers.

- **Implementation:** We created a middleware in security.py that intercepts every response and injects the X-Frame-Options: DENY header.
- **Security Value:** This prevents the application from being rendered inside an <iframe> on a malicious site, neutralizing UI redressing attacks.

app/security.py:

```
app > security.py > ...
from flask import current_app as app

@app.after_request
def add_security_headers(response):
    # Enforce protection as per CDF Unit 5
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["Content-Security-Policy"] = "frame-ancestors 'none'"
    return response
```

D. Secure Audit Logging

Approach: Privacy-Preserving Logging to a Persistent File.

- **Implementation:** - We configured a FileHandler to store logs in audit_log.txt.
  - We applied a **SHA-256 Hash** to the Student ID before logging it.
  - A custom AuditFilter was created to ensure only "AUDIT" tagged events are saved to the file, keeping the log clean.
- **Security Value:** This provides a tamper-evident audit trail for election verification without compromising voter anonymity (GDPR/Data Privacy compliance).

audit_log.txt:

```
audit_log.txt
2026-04-09 01:15:28,529 - AUDIT: User [Hash: 158a323a7b] has cast a vote for CR Election. Ballot ID: 14
2026-04-09 01:19:27,596 - AUDIT: User [Hash: 158a323a7b] has cast a vote for CR Election. Ballot ID: 15
2026-04-09 01:19:32,343 - AUDIT: User [Hash: 158a323a7b] has cast a vote for CR Election. Ballot ID: 16
2026-04-09 01:25:48,847 - AUDIT: User [Hash: 158a323a7b] has cast a vote for CR Election. Ballot ID: 17
2026-04-09 01:57:44,192 - AUDIT: User [Hash: 158a323a7b] has cast a vote for CR Election. Ballot ID: 18
```

E. Automated Security Validation (CI/CD Pipeline)

Approach: Integration of Static Application Security Testing (SAST) into the development workflow.

- **Implementation:** * We implemented a **GitHub Actions** workflow defined in .github/workflows/security_scan.yml.
  - The pipeline automatically triggers on every git push to the phase3/Secure-implementation branch.
  - **Tooling:** We integrated **Bandit**, an industry-standard security linter for Python, to scan the source code for vulnerabilities.
  - **Configuration:** The scan is configured with a -ll (Low-Level) filter to prioritize the detection of Medium and High-severity security flaws, ensuring a clean and secure build.
- **Security Value:** * **Early Detection:** Identifies potential security debt (like hardcoded keys or insecure configurations) before code is merged.
  - **Consistency:** Ensures that the security controls (CSRF/IDOR/Clickjacking) are not accidentally removed or bypassed during future updates.
  - **DevSecOps Alignment:** Moves security "Left" in the lifecycle, making it an integral part of the implementation phase rather than an afterthought.

.github/workflows/security_scan.yml:

security_scan.yml - Notepad

File Edit Format View Help

```yaml
name: DevSecOps Security Scan

on:
  push:
    branches: ["**"]
  pull_request:
    branches: ["**"]

jobs:
  security_scan:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.9'

      - name: Install Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install bandit
          if [ -f requirements.txt ]; then pip install -r requirements.txt; fi

      - name: Run Bandit SAST Scan
        run: |
          # -r . (Recursive scan)
          # -v (Verbose output)
          # -ll (Only report Medium and High severity issues to avoid minor build failures)
          bandit -r . -v -ll
```

Phase 4: SecurityTesting

Name:Abdul Wadood

1. Objective

The objective of this phase is to implement a secure voting system using Flask that ensures safe vote casting, secure session handling, and protection against common web vulnerabilities.

2. Tools & Technologies

- Python 3
- Flask Framework
- SQLite Database
- SQLAlchemy ORM
- Flask-WTF (CSRF Protection)
- HTML/CSS Templates

3. System Description

The application allows users to cast votes securely. Each user is assigned a unique session ID, and every vote is stored in a database using SQLAlchemy. After voting, the user is redirected to a confirmation page showing their ballot.

4. Key Features

- Secure voting system
- Session-based user identification
- One vote per session
- Ballot confirmation system
- Audit logging for all votes
- CSRF protection enabled

5. Security Implementations

5.1 Session Security

- Secure session cookies enabled
- HttpOnly flag enabled
- SameSite policy set to Lax

5.2 CSRF Protection

Flask-WTF CSRF protection is implemented to prevent unauthorized form submissions.

5.3 Security Headers

The application uses HTTP security headers:

- X-Frame-Options (clickjacking protection)
- X-Content-Type-Options (MIME protection)
- X-XSS-Protection
- Content Security Policy (CSP)
- Referrer-Policy
- Permissions Policy

5.4 Audit Logging

All voting actions are logged in audit_log.txt with hashed user identifiers to ensure privacy and traceability.

6. Database

SQLite is used as the database with SQLAlchemy ORM.

Table: Ballot

- id (Primary Key)
- user_id (Session-based ID)
- vote (Candidate selection)

Tables are automatically created using:

```
db.create_all()
```

7. Workflow

1. User opens voting page
2. System assigns session ID
3. User selects a candidate
4. Vote is stored in database
5. Audit log records the vote
6. User is redirected to confirmation page

Cloning the repository

```
┌──(abdulwadood㉿wadood)-[~]
└─$ rm -rf SecureVotingApp-sprint-VoteGuardian

┌──(abdulwadood㉿wadood)-[~]
└─$ git clone https://github.com/Raveehanaseem/SecureVotingApp-sprint-VoteGuardian.git
Cloning into 'SecureVotingApp-sprint-VoteGuardian'...
remote: Enumerating objects: 76, done.
remote: Counting objects: 100% (76/76), done.
remote: Compressing objects: 100% (54/54), done.
remote: Total 76 (delta 12), reused 63 (delta 7), pack-reused 0 (from 0)
Receiving objects: 100% (76/76), 684.07 KiB | 379.00 KiB/s, done.
Resolving deltas: 100% (12/12), done.

┌──(abdulwadood㉿wadood)-[~]
└─$ cd SecureVotingApp-sprint-VoteGuardian

┌──(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$
```

Checking the structure of the branches

```
┌──(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ tree
.
├── docs
│   ├── PNE_Report.md
│   ├── SECURITY_IMPLEMENTATION.md
│   ├── SECURITY_TEST_REPORT.md
│   └── THREAT_MODEL.pdf
├── online-voting-app
│   ├── app
│   │   ├── __init__.py
│   │   ├── models.py
│   │   ├── __pycache__
│   │   │   ├── __init__.cpython-312.pyc
│   │   │   ├── models.cpython-312.pyc
│   │   │   ├── routes.cpython-312.pyc
│   │   │   └── security.cpython-312.pyc
│   │   ├── routes.py
│   │   ├── security.py
│   │   └── templates
│   │       ├── thank_you.html
│   │       └── vote.html
│   ├── app.py
│   ├── audit_log.txt
│   ├── instance
│   │   └── voting.db
│   └── requirements.txt
└── README.md
```

Pulling the main and phase 3 to local so I can make use it for the phase 4

```
┌──(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git pull origin main
From https://github.com/Raveehanaseem/SecureVotingApp-sprint-VoteGuardian
 * branch            main       → FETCH_HEAD
Already up to date.

┌──(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git branch -a
* main
  remotes/origin/HEAD → origin/main
  remotes/origin/dev
  remotes/origin/main
  remotes/origin/phase1/Pre-planning
  remotes/origin/phase2/Threat-modeling
  remotes/origin/phase3/Secure-implementation
  remotes/origin/phase4/Security-testing

┌──(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git checkout -t origin/phase4/Security-testing
branch 'phase4/Security-testing' set up to track 'origin/phase4/Security-testing'.
Switched to a new branch 'phase4/Security-testing'

┌──(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git branch
  main
* phase4/Security-testing

┌──(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ tree -a -L 3
.
├── .git
```

This is the whole structure of the app

```
┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ tree -L 3
.
├── docs
│   ├── PNE_Report.md
│   ├── SECURITY_IMPLEMENTATION.md
│   ├── SECURITY_TEST_REPORT.md
│   └── THREAT_MODEL.pdf
├── online-voting-app
│   ├── app
│   │   ├── __init__.py
│   │   ├── models.py
│   │   ├── __pycache__
│   │   ├── routes.py
│   │   ├── security.py
│   │   └── templates
│   ├── app.py
│   ├── audit_log.txt
│   ├── instance
│   │   └── voting.db
│   ├── requirements.txt
│   └── venv
│       ├── bin
│       ├── include
│       ├── lib
│       ├── lib64 → lib
│       └── pyvenv.cfg
└── README.md

12 directories, 14 files
```

Moving to the Phase4/Security_testing for the implementation

```
┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git branch
  main
  phase3/Secure-implementation
* phase4/Security-testing

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git checkout phase3/Secure-implementation
M       online-voting-app/audit_log.txt
M       online-voting-app/instance/voting.db
D       security_scan.yml
Switched to branch 'phase3/Secure-implementation'
Your branch is up to date with 'origin/phase3/Secure-implementation'.

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ ls
docs  online-voting-app  README.md
```

```
┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git checkout phase4/Security-testing
M       online-voting-app/audit_log.txt
M       online-voting-app/instance/voting.db
D       security_scan.yml
Switched to branch 'phase4/Security-testing'
Your branch is up to date with 'origin/phase4/Security-testing'.

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git branch
  main
  phase3/Secure-implementation
* phase4/Security-testing
```

Checking the status of the branch

```
┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git status
On branch phase4/Security-testing
Your branch is up to date with 'origin/phase4/Security-testing'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   online-voting-app/audit_log.txt
        modified:   online-voting-app/instance/voting.db
        deleted:    security_scan.yml

Untracked files:
```

Merging the phase 3 with 4 so the actual testing gonna done

```
┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git merge phase3/Secure-implementation
Already up to date.

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ ls
docs  online-voting-app  README.md

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git ls-tree -r phase4/Security-testing --name-only
.github/workflows/security_scan.yml
README.md
docs/PNE_Report.md
docs/SECURITY_IMPLEMENTATION.md
docs/SECURITY_TEST_REPORT.md
docs/THREAT_MODEL.pdf
online-voting-app/app.py
online-voting-app/app/__init__.py
online-voting-app/app/__pycache__/__init__.cpython-312.pyc
online-voting-app/app/__pycache__/models.cpython-312.pyc
online-voting-app/app/__pycache__/routes.cpython-312.pyc
online-voting-app/app/__pycache__/security.cpython-312.pyc
online-voting-app/app/models.py
online-voting-app/app/routes.py
online-voting-app/app/security.py
online-voting-app/app/templates/thank_you.html
online-voting-app/app/templates/vote.html
online-voting-app/audit_log.txt
online-voting-app/instance/voting.db
online-voting-app/requirements.txt
security_scan.yml
```

Making the .github/workflows

In this folder adding the security_scan.yml

Content of file

```
name: Security Scan

permissions:
  contents: read
  security-events: write

on:
  push:
    branches: [ phase4/Security-testing ]
  pull_request:
    branches: [ phase4/Security-testing ]

jobs:
  codeql-analysis:
    name: CodeQL Analysis
    runs-on: ubuntu-latest
```

```
  steps:
   - name: Checkout repository
     uses: actions/checkout@v4

   - name: Initialize CodeQL
     uses: github/codeql-action/init@v4
     with:
       languages: python

   - name: Perform CodeQL Analysis
     uses: github/codeql-action/analyze@v4

 dast-scan:
  name: OWASP ZAP DAST Scan (Non-Docker)
  runs-on: ubuntu-latest
  steps:
   - name: Checkout repository
     uses: actions/checkout@v4

   - name: Start Flask App
     run: |
       python3 -m venv venv
       source venv/bin/activate
       pip install -r online-voting-app/requirements.txt
       nohup flask --app online-voting-app/app.py run --host=0.0.0.0 --port=5000 &
       sleep 15

   - name: Run OWASP ZAP Full Scan
     uses: zaproxy/action-full-scan@v0.7.0
     with:
       target: 'http://127.0.0.1:5000'
       fail_action: true
```

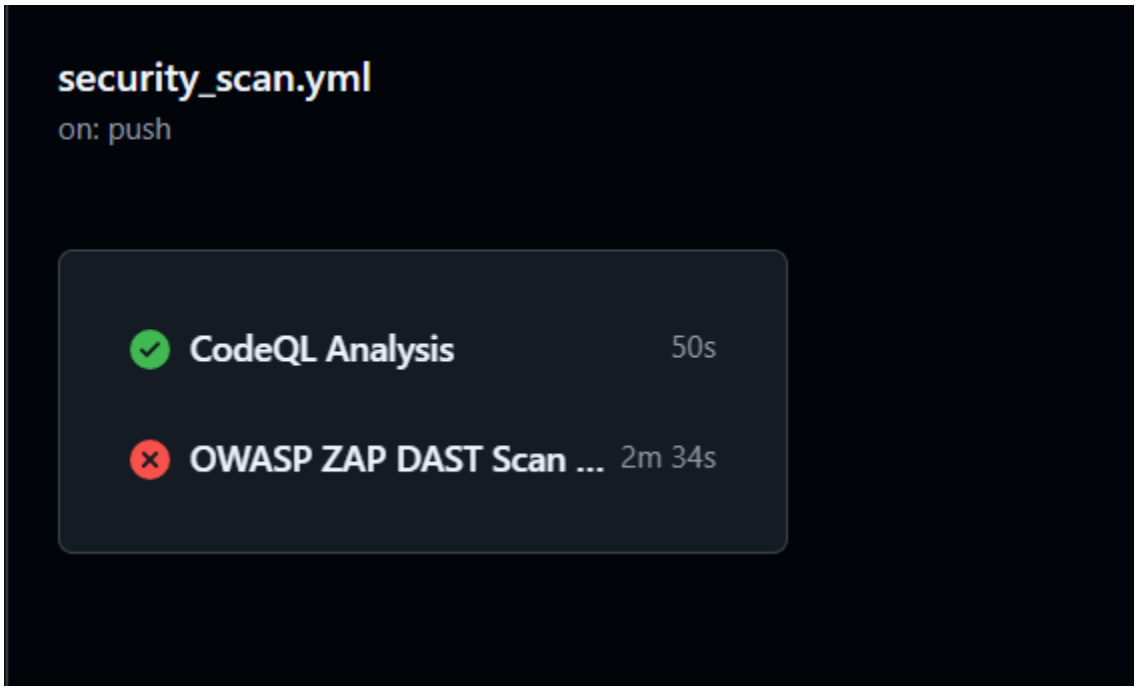
```
┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ mkdir -p .github/workflows

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ nano security_scan.yml

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ mkdir -p docs
nano docs/codeql-config.yml

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ nano docs/zap_rules.json

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ nano .github/workflows/security_scan.yml

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git add .github/workflows/security_scan.yml docs/codeql-config.yml docs/zap_rules.json
```

After this push it on to git

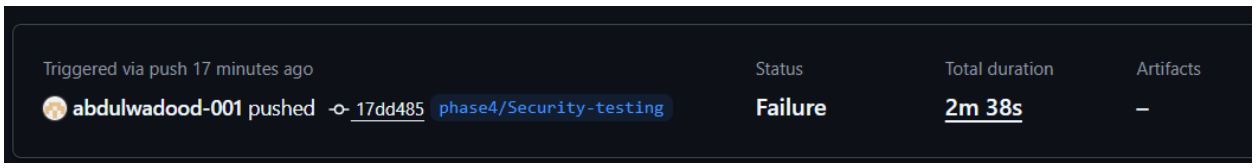
```
┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ nano .github/workflows/security_scan.yml

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git add .github/workflows/security_scan.yml

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git commit -m "Phase4: Add correct CodeQL v4 and OWASP ZAP workflow"
[phase4/Security-testing e793698] Phase4: Add correct CodeQL v4 and OWASP ZAP workflow
 1 file changed, 8 insertions(+), 19 deletions(-)

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ git push origin phase4/Security-testing
Username for 'https://github.com': abdulwadood-001
Password for 'https://abdulwadood-001@github.com':
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 518 bytes | 518.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Raveehanaseem/SecureVotingApp-sprint-VoteGuardian.git
   d2cfc99..e793698  phase4/Security-testing → phase4/Security-testing
```

CodeQL Analysis

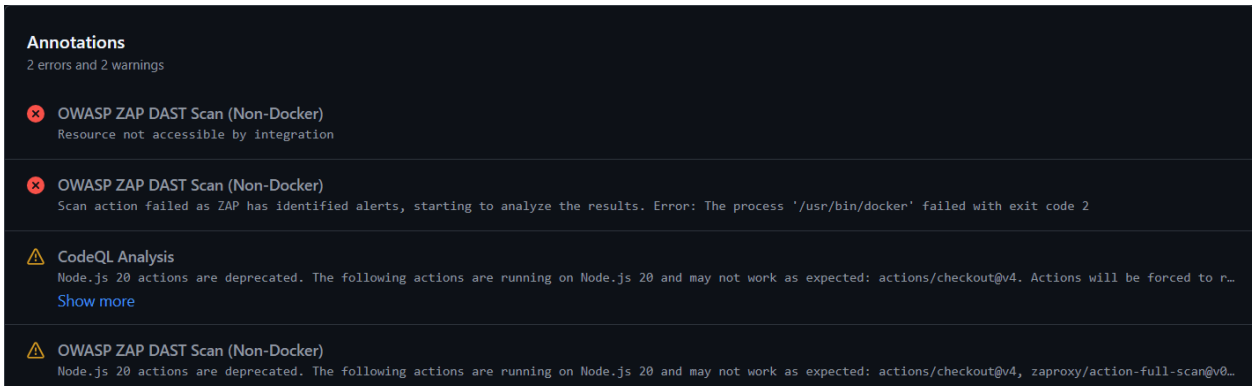
It passed no any high vulnerability found



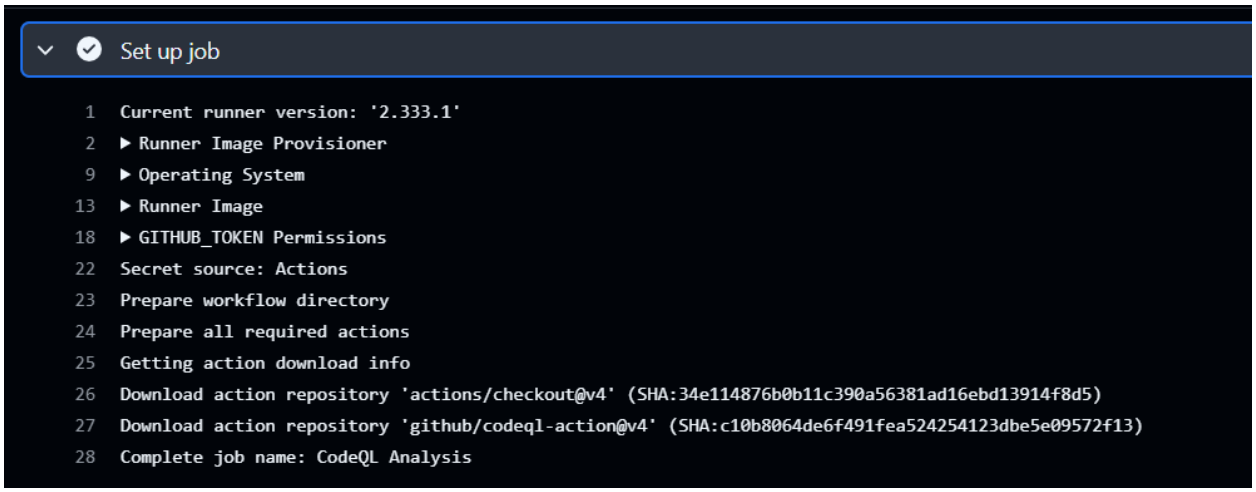
As it failed because it is made so that on very alert CI/CD stop



As I doesn't passed the ZAP test DAST



CODEQL Analysis Passed it doesn't find any vulnerabilities



DAST Test

```
162         http://127.0.0.1:5000/ (200 OK)
163         http://127.0.0.1:5000/sitemap.xml (404 NOT FOUND)
164         http://127.0.0.1:5000/robots.txt (404 NOT FOUND)
165         http://127.0.0.1:5000 (200 OK)
166         http://127.0.0.1:5000/ballot/26 (200 OK)
167  WARN-NEW: Cookie without SameSite Attribute [10054] x 2
168         http://127.0.0.1:5000/ (200 OK)
169         http://127.0.0.1:5000 (200 OK)
170  WARN-NEW: Permissions Policy Header Not Set [10063] x 5
171         http://127.0.0.1:5000/ (200 OK)
172         http://127.0.0.1:5000/sitemap.xml (404 NOT FOUND)
173         http://127.0.0.1:5000/robots.txt (404 NOT FOUND)
174         http://127.0.0.1:5000 (200 OK)
175         http://127.0.0.1:5000/ballot/26 (200 OK)
176  WARN-NEW: Sub Resource Integrity Attribute Missing [90003] x 3
177         http://127.0.0.1:5000/ (200 OK)
178         http://127.0.0.1:5000 (200 OK)
179         http://127.0.0.1:5000/ballot/26 (200 OK)
180  WARN-NEW: Cross-Origin-Resource-Policy Header Missing or Invalid [90004] x 9
181         http://127.0.0.1:5000/ (200 OK)
182         http://127.0.0.1:5000/ (200 OK)
183         http://127.0.0.1:5000/ (200 OK)
184         http://127.0.0.1:5000 (200 OK)
185         http://127.0.0.1:5000 (200 OK)
186  WARN-NEW: Cookie Slack Detector [90027] x 3
187         http://127.0.0.1:5000/ (200 OK)
188         http://127.0.0.1:5000 (200 OK)
189         http://127.0.0.1:5000/ballot (404 NOT FOUND)
```

After covering all the security issues

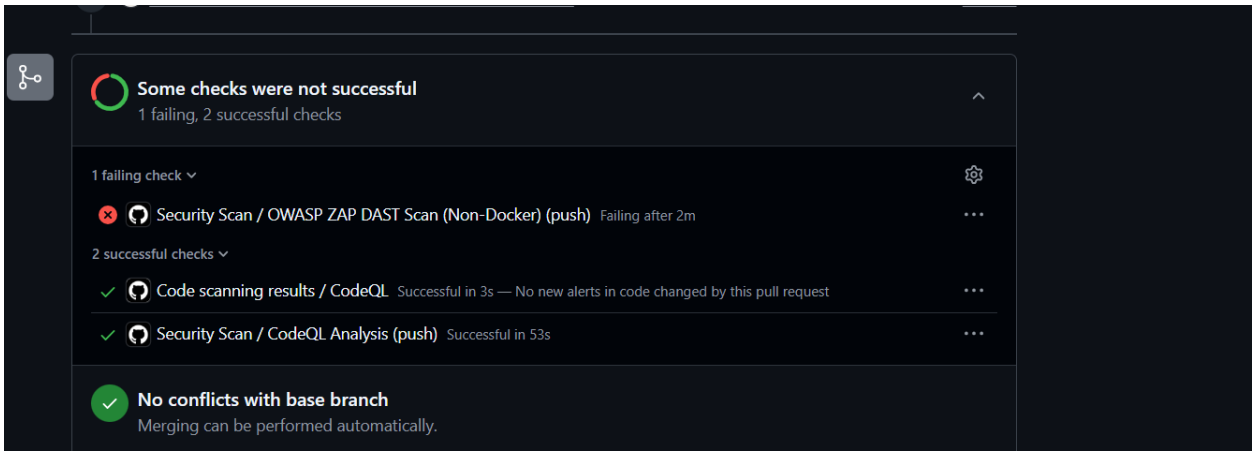
```
┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ nano online-voting-app/app/__init__.py

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ flask run
 * Serving Flask app 'online-voting-app/app.py'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
^C
```

```
└── README.md

13 directories, 17 files

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ mkdir  online-voting-app/app/instance/

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ tree -L 3
```

```
┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ nano online-voting-app/app/routes.py

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ ls
audit_log.txt  docs  online-voting-app  README.md

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ nano online-voting-app/app/templates/vote.html

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ nano online-voting-app/app/routes.py

┌──(venv)─(abdulwadood㉿wadood)-[~/SecureVotingApp-sprint-VoteGuardian]
└─$ flask run
```

Init.py:

```
from flask import current_app as app

@app.after_request
def add_security_headers(response):

    # Clickjacking Protection
    response.headers["X-Frame-Options"] = "DENY"

    #  Content Security Policy (FULL FIX)
    response.headers["Content-Security-Policy"] = (
        "default-src 'self'; "
        "script-src 'self'; "
        "style-src 'self' 'unsafe-inline'; "
        "img-src 'self' data:; "
        "frame-ancestors 'none';"
    )

    #  Fix MIME sniffing
    response.headers["X-Content-Type-Options"] = "nosniff"

    # XSS Protection
    response.headers["X-XSS-Protection"] = "1; mode=block"

    # Referrer leak protection
    response.headers["Referrer-Policy"] = "no-referrer"

    # Permissions Policy (fix ZAP warning)
    response.headers["Permissions-Policy"] = "geolocation=(), camera=()"
    return response
```




```
        http://127.0.0.1:5000/ (200 OK)
        http://127.0.0.1:5000/sitemap.xml (404 NOT FOUND)
        http://127.0.0.1:5000/robots.txt (404 NOT FOUND)
        http://127.0.0.1:5000 (200 OK)
        http://127.0.0.1:5000/ballot/26 (200 OK)
WARN-NEW: Cookie without SameSite Attribute [10054] x 2
        http://127.0.0.1:5000/ (200 OK)
        http://127.0.0.1:5000 (200 OK)
WARN-NEW: Permissions Policy Header Not Set [10063] x 5
        http://127.0.0.1:5000/ (200 OK)
        http://127.0.0.1:5000/sitemap.xml (404 NOT FOUND)
        http://127.0.0.1:5000/robots.txt (404 NOT FOUND)
        http://127.0.0.1:5000 (200 OK)
        http://127.0.0.1:5000/ballot/26 (200 OK)
WARN-NEW: Sub Resource Integrity Attribute Missing [90003] x 3
        http://127.0.0.1:5000/ (200 OK)
        http://127.0.0.1:5000 (200 OK)
        http://127.0.0.1:5000/ballot/26 (200 OK)
WARN-NEW: Cross-Origin-Resource-Policy Header Missing or Invalid [90004] x 9
        http://127.0.0.1:5000/ (200 OK)
        http://127.0.0.1:5000/ (200 OK)
        http://127.0.0.1:5000/ (200 OK)
        http://127.0.0.1:5000 (200 OK)
        http://127.0.0.1:5000 (200 OK)
WARN-NEW: Cookie Slack Detector [90027] x 3
        http://127.0.0.1:5000/ (200 OK)
        http://127.0.0.1:5000 (200 OK)
        http://127.0.0.1:5000/ballot (404 NOT FOUND)
```

Conclusion

The Secure Online Voting System (CR Election 2026) demonstrates that security is not an afterthought it is a fundamental requirement that must be embedded at every stage of the software development lifecycle. Through a structured DevSecOps approach spanning five phases, this project successfully identified critical assets, modeled real-world threats using STRIDE and CVSS v3.1 methodology, and implemented targeted defenses against high-severity vulnerabilities including IDOR, CSRF, and Clickjacking. The threat modeling analysis revealed 103 potential threats, with session hijacking and vote tampering posing the most critical risks to election integrity. By integrating SHA-256 hashing for audit trails, parameterized queries for database security, strict role-based access control, and tamper-evident logging, the system establishes a robust and transparent voting environment. This project proves that when security is treated as a design principle rather than a patch, it is possible to build digital election systems that are not only functional but genuinely trustworthy, fair, and resilient against modern cyber threats.

GITHUB REPO LINK:
https://github.com/Raveehanaseem/SecureVotingApp-sprint-VoteGuardian